D1.3 Selezione delle tecnologie e strategia di implementazione

Sommario

Fase 1 - Apprendimento e training **1**
Componenti soluzione 1
Estensione per Chrome 2
Attivazione e Monitoraggio 2
Tracciamento e Estrazione dei Dati 2
Rilevamento delle API 3
Ricostruzione dell'Intento 3
Manifest 3
Invio dei dati al server Training-data Builder 4
Training Data Builder 5
Tecnologie per il Training-Data Builder **6**
Architettura Node.js 6
Architettura Java 6
Differenze principali 6
Utilizzo di MongoDB 6
Benefici ed estensioni future 7
Fase 2 Running della soluzione **8**
Due scenari operativi differenti 8
Run con API 8
Run con API 8
Use case operativi 8
Use Case 1: Inquiry sulle Fatture di un Cliente 9
Step 1: Ricerca del cliente 9
Step 2: Ricerca delle fatture pagate 9
Caso A: Fatture trovate 9
Caso B: Nessuna fattura trovata 10
Use Case 2: Creazione di una Fattura con Parametro Mancante 10
Soluzione MCP **13**

Questo documento descrive il sistema di tracciamento analisi delle interazioni che un utente ha con sito target, al fine di determinare gli intenti delle azioni dell'utente, che rappresentano l'**obiettivo finale** che un utente vuole raggiungere attraverso una sequenza di azioni all'interno di un'interfaccia o sistema.

Per poter operare in modo efficace in un contesto applicativo reale, il sistema ha bisogno di una fase iniziale di **apprendimento e configurazione**, durante la quale vengono raccolte e strutturate tutte le informazioni necessarie per alimentare sia il **modello di linguaggio (LLM)** sia lo **strato di orchestrazione** rappresentato dal **Model Context Definition (MCD)**.

Questa fase non è un addestramento nel senso strettamente tecnico del fine-tuning, ma è un processo di **modellazione della conoscenza del dominio**, finalizzato a rendere il sistema capace di riconoscere le richieste dell'utente (gli intenti), raccogliere i dati rilevanti per ciascun flusso e invocare correttamente le API o i tool MCP necessari per soddisfare tali richieste.

In concreto, vengono svolte tre attività principali:

1. **Raccolta degli intenti**
   Si parte mappando le esigenze e le azioni che gli utenti possono esprimere in linguaggio naturale. Questi intenti rappresentano gli obiettivi che il sistema deve essere in grado di riconoscere, come ad esempio "verificare una fattura", "creare un nuovo ordine", o "aggiornare un indirizzo di spedizione". Ogni intento va descritto in modo chiaro, associandolo a esempi linguistici e a criteri di disambiguazione.

2. **Identificazione dei dati necessari**
   Per ogni intento, si definiscono i dati che devono essere raccolti per poter eseguire l'azione richiesta. Questi dati, che corrispondono ai parametri dei tool MCP, devono essere identificati, classificati (obbligatori, opzionali, derivabili) e descritti semanticamente. In questa fase si costruiscono anche i dizionari, le ontologie di campo e le regole di validazione che aiuteranno il modello a guidare correttamente l'interazione.

3. **Mappatura delle azioni tecniche (API/tool)**
   Infine, per ogni intento si definisce quale tool MCP (cioè quale API o servizio) deve essere invocato per completare l'azione. Questa mappatura è ciò che alimenta lo **strato MCD**, ovvero la rappresentazione machine-readable della logica operativa del sistema. Il MCD è ciò che consente al client MCP di sapere quali azioni può compiere, come costruire correttamente la richiesta e come interpretare la risposta.

Questa fase iniziale è quindi fondamentale: costituisce la base conoscitiva su cui il sistema costruirà la propria capacità di dialogare in modo efficace e operare in modo affidabile. Una raccolta incompleta o incoerente in questa fase si traduce in risposte ambigue, errori nell'invocazione dei tool o necessità di riformulazione continua da parte dell'utente.

In sintesi, l'apprendimento iniziale è il momento in cui si **trasforma la conoscenza tacita dei processi aziendali in una struttura formalizzata**, interpretabile sia dal modello LLM,

che deve capire cosa vuole l’utente, sia dal layer MCP/MCD, che deve sapere cosa fare e come farlo.

Fase 1 - Apprendimento e training

La soluzione si basa sull'utilizzo di un'estensione per il browser Google Chrome, progettata per registrare le azioni degli utenti sul DOM e le chiamate API effettuate durante la navigazione sul sito. I dati raccolti vengono anonimizzati localmente e inviati a un server centrale, denominato "Training Data Builder", per la successiva elaborazione e classificazione degli intenti utente.

L'obiettivo è fornire uno strumento utile per comprendere i comportamenti degli utenti, supportare attività di testing e QA, migliorare l'usabilità del software e costruire dataset utili per applicazioni di machine learning.

Un **intento** rappresenta l'obiettivo finale che un utente cerca di raggiungere tramite una sequenza di azioni sul sito. Alcuni esempi di intenti comuni includono l'invio di una fattura, l'aggiornamento di un'anagrafica cliente, l'esportazione di un preventivo in PDF e la creazione di un nuovo prodotto. L'intento viene dedotto analizzando la sequenza di eventi utente che culminano in una specifica chiamata API.

La fase di apprendimento permette di raccogliere le informazioni necessarie per classificare gli intenti e fornire le istruzioni ai componenti della fase Running. Lo scopo è che il modello LLM sia in grado di interagire con l’utente, tenendo conto del dominio applicativo e delle azioni determinate dagli intenti. E’ fondamentale che

Componenti soluzione

L’apprendimento è gestito da due componenti distinti, un addon per il browser (Chrome) ed un server applicativo per l’elaborazione (vedi figura 1).

L'estensione viene attivata esclusivamente sul dominio target. Al suo interno, include funzionalità per la raccolta di eventi utente, come click, input e selezioni di menu. Intercetta inoltre le chiamate API effettuate dalla pagina, ricostruisce il flusso utente a partire da un punto di attivazione e applica un processo di anonimizzazione ai dati raccolti. Una volta elaborati, i dati vengono inviati al server Training Data Builder.

Il server riceve i log delle interazioni utente inviati in formato JSON. Una volta ricevuti, analizza le sequenze di azioni per identificare l'intento sottostante. I flussi vengono salvati in un database a fini di analisi, formazione di modelli di machine learning o validazione manuale. Il server può anche opzionalmente restituire etichette o conferme all'estensione client.

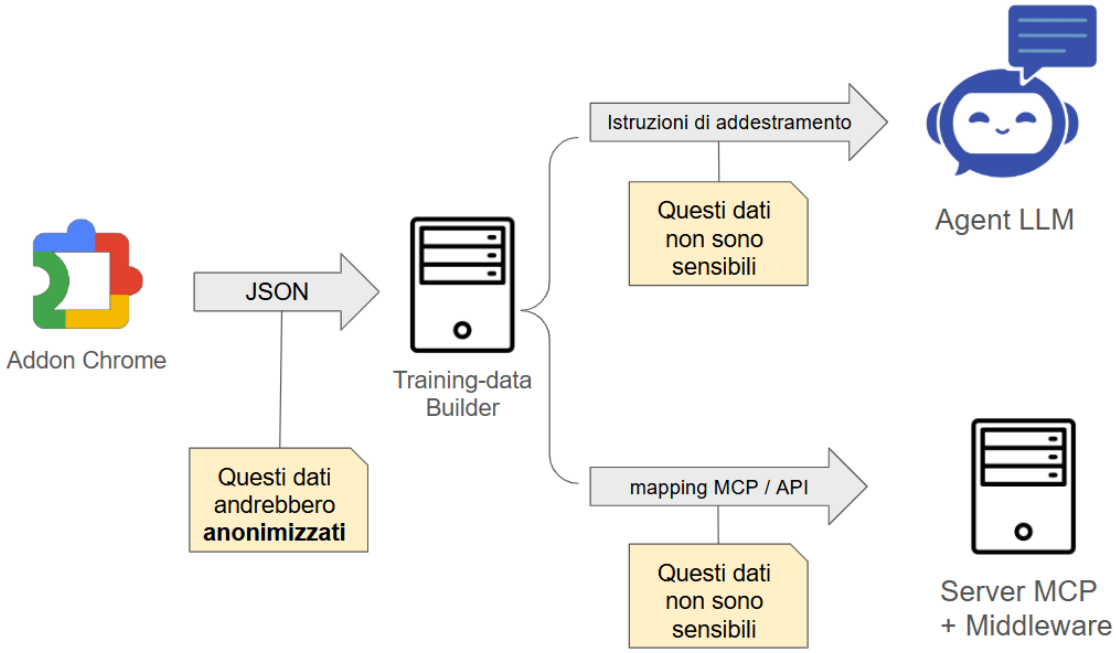


Figura 1 - Componenti soluzione

Estensione per Chrome

Il progetto ha lo scopo di sviluppare **un'estensione per Google Chrome** capace di tracciare le azioni dell'utente sul sito target (nell'esempio www.software-fatturazione.it), raccogliere i dati rilevanti e inviarli a un server centrale denominato **Training-Data Builder**. Questo server ha il compito di elaborare i dati ricevuti per identificare gli intenti dell'utente e costruire un dataset utile per analisi, miglioramenti UX e automazione.

Attivazione e Monitoraggio

L'estensione è progettata per funzionare esclusivamente all'interno del dominio target – in questo caso, `www.software-fatturazione.it` – garantendo che le sue funzionalità siano attivate solo nel contesto operativo dell'applicazione di fatturazione. L'attivazione avviene in modo automatico quando l'utente accede a URL specifici, come ad esempio `/fattura/nuova`, che indicano l'inizio di un'azione rilevante come la creazione di una nuova fattura. Anche l'interazione con elementi significativi dell'interfaccia, come un pulsante etichettato "Crea Fattura", viene interpretata come un segnale di attivazione. Questo approccio selettivo permette all'estensione di operare con efficienza, intervenendo solo nei momenti in cui è realmente necessario, riducendo il carico computazionale e aumentando la precisione del tracciamento.

Tracciamento e Estrazione dei Dati

Una volta attivata, l'estensione avvia un monitoraggio dettagliato di tutte le interazioni dell'utente. Vengono registrati clic su voci di menu, pulsanti, cambi di sezione e soprattutto

l'inserimento di dati nei campi del modulo. Durante la fase di compilazione, l'estensione è in grado di estrarre in modo puntuale le informazioni fornite dall'utente, mantenendo al contempo il rispetto della privacy grazie alla pseudonimizzazione. Questa fase di estrazione è essenziale per contestualizzare l'azione in corso e raccogliere elementi utili all'identificazione dell'intento. La qualità e la coerenza di questo tracciamento sono fondamentali per la successiva analisi dei flussi utente.

Rilevamento delle API

Un aspetto centrale della soluzione è la capacità di intercettare le chiamate API inviate dal browser, consentendo di monitorare la comunicazione tra frontend e backend. In particolare, vengono identificate le richieste che rappresentano l'esecuzione concreta di un'azione intenzionale da parte dell'utente. Ad esempio, una richiesta `POST` verso l'endpoint `/api/fattura/invia` viene riconosciuta come il completamento del processo di invio di una fattura. Questo meccanismo permette di distinguere le interazioni preliminari da quelle che confermano effettivamente un intento, offrendo una validazione robusta del comportamento utente osservato.

Ricostruzione dell'Intento

Dal momento dell'attivazione fino alla chiamata API che conferma l'intento, l'estensione registra in sequenza tutte le azioni compiute dall'utente. Questo insieme di eventi costituisce il cosiddetto "percorso utente", che documenta in modo strutturato l'esperienza dell'utente nell'interfaccia: URL visitati, interazioni, dati inseriti e conferme finali. Ogni percorso viene poi associato a un intento specifico, dedotto dalla chiamata API finale. Nel caso della richiesta `POST /api/fattura/invia`, l'intero flusso che ha portato a quella chiamata viene etichettato come "Inviare fattura". Questa classificazione consente di analizzare non solo le azioni intraprese, ma anche la logica con cui l'utente le ha eseguite, fornendo insight preziosi per l'ottimizzazione dell'esperienza, il miglioramento dei flussi e l'identificazione di eventuali ostacoli o inefficienze.

Manifest

Durante l'interazione dell'utente con la pagina, l'estensione registra tutte le azioni rilevanti. In particolare, i click sugli elementi dell'interfaccia HTML e i cambiamenti nei campi di input vengono monitorati e archiviati. Parallelamente, il background script intercetta le chiamate API effettuate dalla pagina.

```JSON
{
  "manifest_version": 3,
```

```
  "name": "software-fatturazione.it Intent Tracker",
  "version": "1.0",
  "description": "Traccia azioni utente e chiamate API per ricostruire gli intenti.",
  "permissions": [
    "storage",
    "tabs",
    "scripting",
    "webRequest",
    "webRequestBlocking"
  ],
  "host_permissions": [
    "https://www.software-fatturazione.it/*"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [
    {
      "matches": ["https://www.software-fatturazione.it/*"],
      "js": ["content.js"],
      "run_at": "document_idle"
    }
  ],
  "action": {
    "default_popup": "popup.html",
    "default_title": "Intent Tracker"
  }
}
```

Questo `manifest.json` per l'addon Chrome specifica che:

- Si attiva su **tutte le pagine del sito software-fatturazione.it**
- Usa **un service worker (background.js)** per monitorare le chiamate API
- Inietta uno **script di contenuto (content.js)** per tracciare click, input e menu
- Usa **permessi** per intercettare traffico di rete (`webRequest`)
- Include un **popup** per visualizzare il log

Invio dei dati al server Training-data Builder

Una volta riconosciuto un intento, il percorso utente viene anonimizzato e preparato per l'invio. Il payload viene inviato in formato JSON strutturato. Ecco un esempio:

JSON

```
{
  "userId": "test-pc-07",
  "timestamp": "2025-06-15T12:45:00Z",
  "url": "https://www.software-fatturazione.it/fattura/nuova",
```

```
  "events": [
    { "type": "click", "element": "button#nuova-fattura", "time": 0 },
    { "type": "input", "field": "cliente", "value": "anon_42a8fc23", "time":
5 },
    { "type": "click", "element": "button#invia", "time": 9 },
    { "type": "api", "url": "/api/fattura/invia", "method": "POST", "time":
10 }
  ]
}
```


L'estensione applica una funzione di pseudonimizzazione deterministica per tutti i dati sensibili. Se l'input è un'email, viene diviso in local part e dominio. La parte locale viene sostituita con un identificativo coerente e mascherato, mentre il dominio viene sostituito con un dominio fittizio corrispondente. Se l'input è una stringa generica (come un nome o un codice), viene generato un valore pseudonimo alfanumerico stabile nel tempo. Tutti i valori trasformati vengono registrati in una mappa in memoria per garantire coerenza durante tutta la sessione. La struttura e il pattern dell'input originale vengono preservati.

L'estensione è progettata per operare esclusivamente sul dominio autorizzato (nell'esempio www.software-fatturazione.it). I dati sensibili vengono anonimizzati prima della trasmissione e inviati solo tramite connessioni HTTPS. È possibile integrare un meccanismo opzionale di autenticazione basato su token per proteggere ulteriormente l'endpoint del server ricevente.

Training Data Builder

Il **Training Data Builder** è il componente server-side della soluzione progettata per il tracciamento e l'analisi degli intenti utente. Il suo ruolo principale è raccogliere, archiviare ed elaborare i flussi di interazione generati dall'estensione Chrome installata sui browser degli utenti. Questi flussi – registrati sotto forma di eventi strutturati (click, input, chiamate API) – rappresentano il percorso compiuto dall'utente all'interno dell'applicazione web e vengono arricchiti con metadati e pseudonimizzazione locale prima di essere inviati al server.

Il sistema è progettato per operare come un **aggregatore e normalizzatore di dati comportamentali**, in grado di inferire automaticamente gli intenti a partire dai log delle sessioni, supportando così attività di validazione manuale, costruzione di dataset supervisionati e addestramento di modelli di machine learning. Grazie all'integrazione con un database documentale (MongoDB), il Training Data Builder è in grado di gestire dati flessibili e di grande volume, assicurando al tempo stesso accessibilità e scalabilità.

Questa componente rappresenta il punto di convergenza tra l'osservazione locale (client) e l'analisi centralizzata (server), fornendo la base per lo sviluppo di funzionalità avanzate quali automazione dei test, analytics comportamentale, rilevamento di anomalie e ottimizzazione dell'esperienza utente.

Tecnologie per il Training-Data Builder

La soluzione client si basa sull'utilizzo di tecnologie compatibili con l'ecosistema delle estensioni di Chrome, in particolare con le specifiche Manifest V3. Sul lato server, sono state prese in considerazione due architetture principali: una basata su Node.js e una su Java.

Architettura Node.js

L'architettura Node.js si fonda sull'utilizzo del framework Express.js, sviluppato in JavaScript e pensato per offrire un approccio semplice e modulare alla creazione di API REST. Il server riceve i dati in formato JSON e li gestisce tramite Express, mentre MongoDB viene utilizzato come database non relazionale per la persistenza dei flussi utente. Questo approccio è particolarmente vantaggioso per i team con competenze JavaScript, grazie alla sintonia tra estensione client e server. Il setup è rapido, l'ecosistema è molto esteso e consente una prototipazione veloce, rendendolo ideale per microservizi, applicazioni leggere o MVP.

Architettura Java

L'architettura Java è costruita utilizzando Spring Boot, in combinazione con Spring Web per la gestione delle API REST e Spring Data MongoDB per l'integrazione con il database. Il linguaggio di programmazione è Java, fortemente tipizzato, e l'ambiente è progettato per offrire scalabilità, struttura e sicurezza. I dati sono gestiti in formato JSON e serializzati/deserializzati tramite il modulo Jackson. Questa soluzione si adatta particolarmente bene a contesti enterprise, ambienti con infrastruttura Java consolidata, e progetti che richiedono governance rigorosa, testing approfondito e possibilità di crescita a lungo termine.

Differenze principali

Le due architetture presentano caratteristiche distinte che ne determinano l'idoneità a diversi contesti. Node.js con Express si distingue per una curva di apprendimento più accessibile, velocità di sviluppo elevata e una predisposizione verso ambienti leggeri e dinamici. Al contrario, Java con Spring Boot richiede maggiori competenze tecniche iniziali, ma offre robustezza, strumenti di integrazione avanzati e un'infrastruttura adatta ad applicazioni complesse e mission-critical. Entrambe le soluzioni utilizzano MongoDB per la gestione dei dati, ma si differenziano per l'approccio allo sviluppo, alla gestione del codice e al modello di distribuzione.

Utilizzo di MongoDB

MongoDB è il database scelto per la memorizzazione dei flussi utente grazie alla sua natura documentale e alla capacità di gestire dati semi-strutturati in formato JSON. Ogni sessione utente viene salvata come un documento autonomo all'interno di una collezione, facilitando operazioni di ricerca, filtro e aggregazione. Questa scelta consente al sistema di registrare flussi anche molto diversi tra loro senza vincoli rigidi di schema, rendendo l'infrastruttura adatta a scenari evolutivi e dinamici. Inoltre, MongoDB supporta in modo nativo operazioni

su dati annidati, come array di eventi, e permette di costruire query complesse per individuare pattern, frequenze e anomalie nei comportamenti degli utenti. Infine, la scalabilità orizzontale di MongoDB consente di supportare facilmente l’aumento del volume di dati in ambienti reali multi-macchina.

Benefici ed estensioni future

Questa architettura consente l'utilizzo distribuito su più postazioni di lavoro, ad esempio per scopi di QA o testing. La raccolta centralizzata e coerente dei dati utente consente di effettuare analisi comportamentali, migliorare l'esperienza utente e generare dataset supervisionati per modelli predittivi. Inoltre, permette di diagnosticare eventuali problemi nei flussi di interazione e di testare nuove funzionalità tramite replay controllati.

Tra le possibili estensioni future si includono la persistenza della mappa di pseudonimizzazione su storage locale per garantire continuità oltre la singola sessione, l'integrazione con un'interfaccia per la validazione manuale degli intenti, la possibilità di riprodurre automaticamente flussi utente a fini di test, e l'interfacciamento con sistemi di ticketing o monitoraggio degli errori.

JSON per l’addestramento LLM

I file JSON raccolti definiscono una struttura dichiarativa pensata per descrivere in modo formale gli **intenti conversazionali** che un modello di linguaggio (LLM) deve essere in grado di riconoscere e gestire all’interno di un sistema integrato tramite MCP (Model Context Protocol). Ogni intento rappresenta un’azione specifica che l’utente può esprimere in linguaggio naturale, come la creazione di una fattura, la consultazione di movimenti contabili o la ricerca di documenti in un determinato intervallo temporale.

La struttura di ciascun intento include diversi elementi fondamentali:

- Una sezione `description`, che raccoglie esempi o formulazioni equivalenti utilizzabili dall’utente per esprimere quell’obiettivo.

- Un blocco `required_fields`, in cui vengono elencati i dati necessari per completare l’operazione. Ogni campo è descritto con il tipo atteso (`str`, `float`, `date`, `int`), una breve etichetta, e la fonte del dato, che può essere l’utente stesso (`user`), una fonte esterna (`api` o `external-db`) o un’entità semantica nota al sistema. Alcuni campi possono includere riferimenti a strumenti MCP da invocare per il recupero dei dati o a entità di dominio rilevanti, come nel caso del cliente.

Ove presente, la chiave `target-tool` indica quale tool MCP deve essere chiamato per eseguire l’azione, una volta che tutti i dati richiesti sono stati raccolti. Inoltre, la sezione `errors` permette di associare codici di errore specifici a messaggi semantici, così che il modello possa gestire in modo appropriato eventuali risposte negative dal backend.

Complessivamente, questa struttura fornisce al sistema una mappa semantica coerente tra **linguaggio naturale, logica applicativa e strumenti operativi**, permettendo al modello di linguaggio di interagire in modo preciso e guidato con i servizi aziendali, sia per rispondere a richieste informative, sia per assistere o eseguire operazioni dispositive.

Esempio JSON per la creazione di fatture

JSON

```
{
  "intents": {
    "create_invoice": {
      "description": [
        "Crea una fattura di vendita",
        "Crea una fattura per un cliente",
        "Crea una fattura del ciclo attivo"
```

```
      ],
      "required_fields": {
        "amount": {
          "type": "float",
          "description": "importo",
          "source": "user"
        },
        "date": {
          "type": "date",
          "description": "data",
          "source": "user"
        },
        "description": {
          "type": "str",
          "description": "descrizione",
          "source": "user"
        },
        "clientId": {
          "type": "int",
          "description": "cliente",
          "source": "api",
          "tool": "/customer",
          "inputType": "string",
          "outputType": "list"
        },
      },
      "target-tool": "/crete-invoice",
      "errors": {
        "001": "data fattura non valida.",
        "002": "ID del cliente sospeso."
      }
    }
  }
}
```

Esempio JSON per la visualizzazione delle fatture di vendita di un periodo

JSON

```
{
  "intents": {
    "search_outgoing_invoices": {
      "description": [
        "elenca le fatture di vendita in un dato periodo",
        "Visualizza le fatture del ciclo attivo di un dato periodo"]
      "required_fields": {
        "start_date": {
```

```
          "type": "date",
          "description": "a partire da",
          "source": "user"
        },
        "end_date": {
          "type": "date",
          "description": "fino a",
          "source": "user"
        }
      },
      "target-tool": "/get-sell-invoices",
      "errors": {
        "001": "range di date non valido.",
        "010": "nessun elemento trovato."
      }

    }
  }
}
```

Esempio JSON per la visualizzazione delle fatture di vendita di un cliente

JSON

```
{
  "intents": {
    "search_outgoing_invoices": {
      "description": "Cerca fatture in uscita per un cliente in un periodo",
      "required_fields": {
        "client_name": {
          "type": "str",
          "description": "cliente",
          "entity": "user"
        },
        "start_date": {
          "type": "date",
          "description": "dal giorno",
          "source": "user"
        },
        "end_date": {
          "type": "date",
          "description": "al giorno",
          "source": "user"
        }
      },
```

```
      "target-tool": "/get-sell-invoices",
      "errors": {
        "001": "range di date non valido.",
        "002": "cliente non trovato.",
        "010": "nessun elemento trovato."

  }
}
```

Fase 2 - Running della soluzione

Il modello AI non comunica direttamente con i servizi applicativi, ma lo fa attraverso un **client MCP**, che gestisce la comunicazione e l'accesso a un **server MCP**. Questo server espone un insieme di strumenti (tool) formalmente descritti, ciascuno dei quali rappresenta una funzione ben definita che il modello può invocare. In questo modo, le operazioni disponibili sono chiaramente documentate e accessibili in maniera uniforme, indipendentemente dal tipo di backend sottostante.

Soluzione basata su MCP

Dal punto di vista del **Model Context Protocol (MCP)**, la soluzione si basa su un'architettura che mette al centro l'interazione strutturata tra il modello di linguaggio e i sistemi informativi. MCP agisce come uno strato di interfaccia tra il mondo conversazionale del modello AI e le funzionalità operative esposte da servizi esterni, siano essi moderni o legacy.

La responsabilità del **client MCP** è orchestrare il dialogo tra il modello e il server: riceve l'intento generato dal modello, lo mappa sul tool appropriato, e lo invoca con i parametri richiesti. Il server MCP esegue la richiesta e restituisce una risposta strutturata, che il modello può poi interpretare e tradurre in linguaggio naturale, oppure utilizzare per guidare ulteriori azioni.

Questa struttura ha diversi vantaggi. Permette di **separare in modo netto la logica conversazionale dalla logica applicativa**, rendendo più facile mantenere, aggiornare o estendere le funzionalità disponibili. Inoltre, poiché i tool sono descritti in modo dichiarativo (tramite un manifest), il modello può conoscere a priori quali campi sono obbligatori, quali tipi di dati si aspettano, e quale output potrà ricevere, senza dover interrogare dinamicamente ogni volta il server.

Dal punto di vista dell'esperienza utente, il protocollo consente di gestire sia interazioni puramente informative (in cui il modello risponde direttamente sulla base dell'invocazione di un tool), sia interazioni più guidate, in cui il modello può accompagnare l'utente nella compilazione di dati o nella conferma di azioni, soprattutto nei casi dispositivi.

In sintesi, MCP fornisce una struttura chiara, estendibile e sicura per abilitare i modelli AI a operare in contesti applicativi reali, mantenendo una netta distinzione tra ciò che è linguaggio e ragionamento da una parte, e ciò che è esecuzione e integrazione tecnica dall'altra.

A titolo di esempio, riportiamo gli step illustrati nella figura:

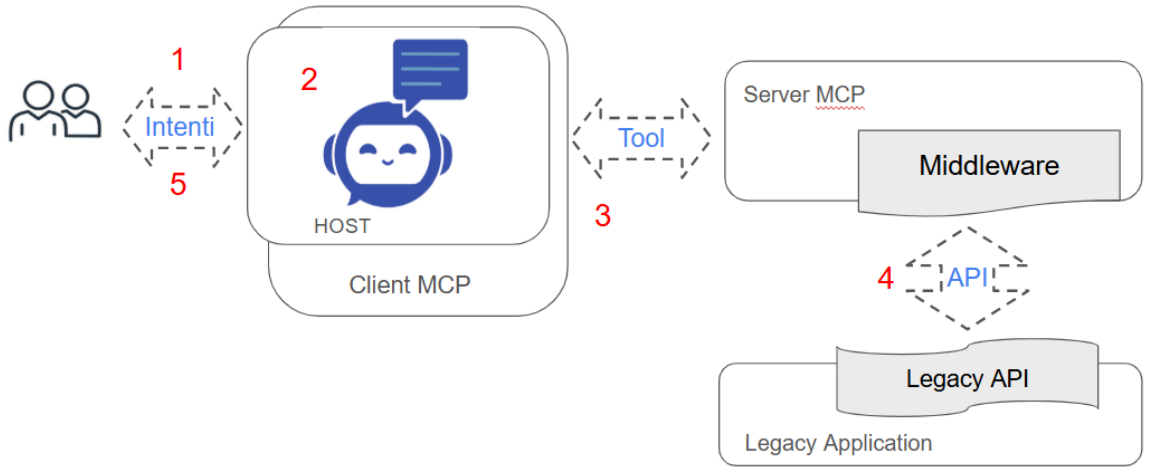
1. L'utente chiede: "Voglio sapere se la fattura 0035 è stata pagata."

2. Il modello AI rileva l'intento: `verificare_stato_fattura`.

3. Il client MCP mappa l'intento su un tool specifico: `check_invoice_status(invoice_id="0035")`.

4. Il server MCP esegue il tool e restituisce la risposta.

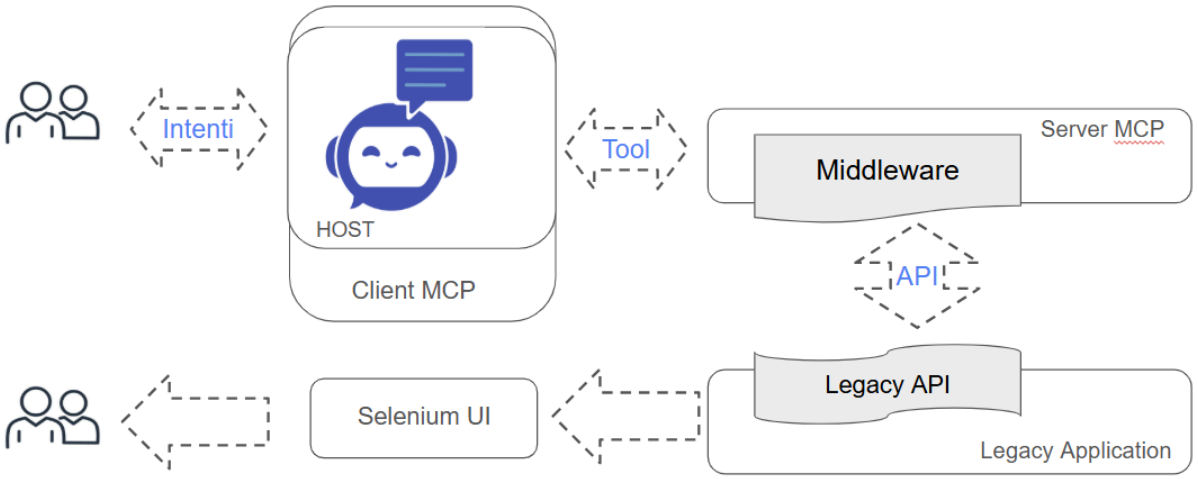
5. Il modello interpreta il risultato e risponde all'utente in linguaggio naturale.



Lo scenario descritto in questo paragrafo rappresenta un'interazione in cui è il **chatbot** (guidato dal modello AI) a fornire direttamente una risposta all'utente. Questo tipo di flusso è particolarmente adatto per le **inquiry**, ovvero richieste informative.

Run con Interazione UI

Prendiamo in esame un secondo scenario operativo, in cui il chatbot **guida l'utente nella navigazione della UI**, ad esempio compilando i campi di una form e attendendo che sia l'utente a confermare o completare l'azione.



Questo tipo di interazione è più adatto per le **azioni dispositive**, ovvero operazioni che modificano lo stato del sistema (come la creazione, modifica o cancellazione di un oggetto).

In questo caso, il modello non esegue direttamente l'azione, ma supporta l'utente passo dopo passo nella sua esecuzione attraverso l'interfaccia.

Esempio di interazione con la UI

Questo esempio rappresenta un caso di interazione automatizzata con l'interfaccia utente (UI), in cui il sistema, tramite un'estensione browser (ad esempio in ambiente Chrome), è in grado di compilare e inviare automaticamente una form web.

JavaScript

```
// Esempio di manifest
{
  "manifest_version": 3,
  "name": "Compilazione GENHAPI",
  "version": "1.0",
  "permissions": ["scripting", "activeTab"],
  "action": {
    "default_popup": "popup.html"
  },
  "background": {
    "service_worker": "automation.js"
  }
}
```

Il frammento JSON mostra un manifest di estensione conforme allo standard `manifest_version: 3`, che definisce le autorizzazioni necessarie (`scripting`, `activeTab`), specifica la risorsa HTML utilizzata per il popup e configura uno script di background (`automation.js`) incaricato di eseguire l'automazione. Lo script riportato nel secondo blocco simula il comportamento dell'utente compilando i campi della form — come il nome del cliente, l'indirizzo, la partita IVA e l'importo — e attivando infine il pulsante di invio (`inviaFattura`).

JavaScript

```
// Esempio di codice di automazione
document.getElementById("cliente").value = "Mario Rossi";
document.getElementById("indirizzo").value = "Via Roma 123, Milano";
document.getElementById("partitaIVA").value = "12345678901";
document.getElementById("importo").value = "350.00";
document.getElementById("inviaFattura").click();
```

Questo tipo di approccio è particolarmente utile nei casi in cui il modello AI non può interagire direttamente con un'API, ma può invece agire sull'interfaccia tramite scripting,

guidando o eseguendo azioni dispositive per conto dell'utente. In questo contesto, la UI diventa il vettore di esecuzione e l'automazione è uno dei meccanismi chiave per implementare interazioni "ibride" tra intelligenza artificiale e sistemi legacy o applicazioni senza endpoint di integrazione.

Use case sul sistema di fatturazione

In questa sezione analizziamo diversi **casi d'uso (use case)** con l'obiettivo di illustrare, attraverso esempi concreti, **come cambia l'interazione tra utente, modello AI e sistema** a seconda del tipo di richiesta.

Ogni scenario evidenzia un diverso approccio conversazionale — ad esempio, una semplice richiesta informativa (inquiry) o un'azione dispositiva — mostrando **quali componenti entrano in gioco** (modello, client MCP, server MCP, interfaccia utente) e **quale ruolo ha ciascun attore** nella gestione della conversazione.

Questa analisi ci aiuta a comprendere meglio **quando e come il modello AI può agire direttamente**, e quando invece deve **delegare o assistere l'utente** nel completamento dell'operazione tramite l'interfaccia.

Use Case 1: Inquiry sulle Fatture di un Cliente

Questo use case mostra come un sistema AI integrato con MCP può gestire richieste utente complesse in più fasi, riconoscere parametri mancanti in base al manifest, coinvolgere l'utente in modo efficace e adattarsi sia a risultati nulli sia a casi di successo, mantenendo la conversazione coerente e utile.

L'utente chiede: *"Quali fatture ho pagato a Rossi quest'anno?"*

Step da eseguire:

1. Cercare il cliente "Rossi".
2. Gestire eventuali ambiguità (più clienti trovati).
3. Recuperare le fatture **pagate** nell'anno corrente per il cliente selezionato.

Step 1: Ricerca del cliente

Tool: `search_customer` **Parametri:** `{ "name": "Rossi" }`

Risposta del server MCP:

```
None
{
  "output": [
    { "customer_id": "9876", "name": "Mario Rossi", "email":
"m.rossi@example.com" },
    { "customer_id": "1021", "name": "Rosa Rossi", "email":
"rosa.rossi@example.com" }
  ]
}
```

Interazione del modello con l'utente:

> "Ho trovato più clienti con il cognome 'Rossi':\n1. Mario Rossi (m.rossi@example.com)\n2. Rosa Rossi (rosa.rossi@example.com)\nQuale intendi? Rispondi con 1 o 2."

Scelta dell'utente: 1 (Mario Rossi)

Step 2: Ricerca delle fatture pagate

Tool: `list_invoices_by_customer` **Parametri:** `{ "customer_id": "9876", "year": 2025, "status": "pagata" }`

Caso A: Fatture trovate

Risposta del server MCP:

```
None
{
  "output": [
    {
      "invoice_id": "INV-2025-0032",
      "amount": 120.0,
      "due_date": "2025-02-15",
      "status": "pagata"
    },
    {
      "invoice_id": "INV-2025-0104",
```

```
      "amount": 250.0,
      "due_date": "2025-05-10",
      "status": "pagata"
    }
  ]
}
```

Risposta del modello all'utente:

> "Ecco le fatture pagate per Mario Rossi nel 2025:\n- INV-2025-0032: 120€, scadenza 15 febbraio\n- INV-2025-0104: 250€, scadenza 10 maggio"

Caso B: Nessuna fattura trovata

Risposta del server MCP:

```
None
{
  "output": []
}
```

Risposta del modello all'utente:

> "Mario Rossi non ha fatture pagate registrate per il 2025. Vuoi vedere anche quelle non ancora pagate o in attesa?"

Use Case 2: Creazione di una Fattura con Parametro Mancante

Scenario Utente:

> "Crea una fattura da 180 euro per il cliente 1234."

Tool: `create_invoice` **Parametri richiesti:** `customer_id` (string), `amount` (number), `due_date` (string)

Il modello AI conosce la lista dei campi richiesti dal tool tramite il manifest MCP e nota che **manca il campo **``.

Interazione del modello con l'utente:

"Per creare la fattura ho bisogno anche della data di scadenza. Puoi indicarmela?"

Risposta dell'utente:

"30 giugno 2025"

Chiamata finale del modello:

```
None
{
  "tool_calls": [
    {
      "id": "call_create_invoice_001",
      "function": {
        "name": "create_invoice",
        "arguments": "{ \"customer_id\": \"1234\", \"amount\":
180.0, \"due_date\": \"2025-06-30\" }"
      },
      "type": "function"
    }
  ]
}
```

Risposta del server MCP:

```
None
{
  "jsonrpc": "2.0",
  "id": "call_create_invoice_001",
  "result": {
    "output": {
      "invoice_id": "INV-2025-0470",
      "status": "creata",
      "created_at": "2025-06-12T17:22:00Z"
    }
  }
}
```

Risposta del modello all'utente:

"Fattura creata con successo per il cliente 1234. ID: INV-2025-0470, importo: 180€, scadenza: 30 giugno 2025."